

Mid-Term Group Portfolio/Project – 2 -2026

Multi-Objective Capacitated Facility Location Problem Using MOEAs

Objective: Develop and implement 02 (two) Multi-Objective Evolutionary Algorithms (MOEAs) in Python to solve the Capacitated Facility Location Problem (CFLP). The problem is to decide which candidate facilities to open and how to assign customers to the opened facilities. The two conflicting objectives are to minimize (1) total facility-opening cost and (2) total customer-allocation cost. You can choose any two MOEAs discussed in the lectures, but must evaluate both algorithms under the same experimental conditions and compare their performance statistically.

- **Group Policy:** Pre-approved groups only. Submissions outside the group list will NOT be accepted.
- **Deadline:** October 08th (Thursday) at 12:00 (midday).
- **Submission Channel:** Canvas.
- **Academic Integrity:** Plagiarism is handled per OsloMet policy. You can find more about it at <https://student.oslomet.no/en/cheating>
- **Use of Artificial Intelligence (AI):** You may use AI tools for coding as permitted by university guidelines, but NO AI tools are permitted to write the report. You can find the OsloMet policy at: <https://student.oslomet.no/en/veiledning-kunstig-intelligens-studentoppgaver>
- **Submission Procedure:** Every group must submit only ONE submission by any group member. We will download the code immediately after the submission deadline to prevent later updates.

The Multi-Objective Capacitated Facility Location Problem (CFLP)

A company wants to serve a set of geographically distributed customers using a subset of candidate warehouses or service facilities. Opening a facility incurs a fixed cost. Each opened facility has a limited capacity, while each customer has a known demand and an allocation/transportation cost for being served from each facility. A feasible solution must open sufficient capacity and assign every customer to exactly one opened facility without exceeding any facility's capacity.

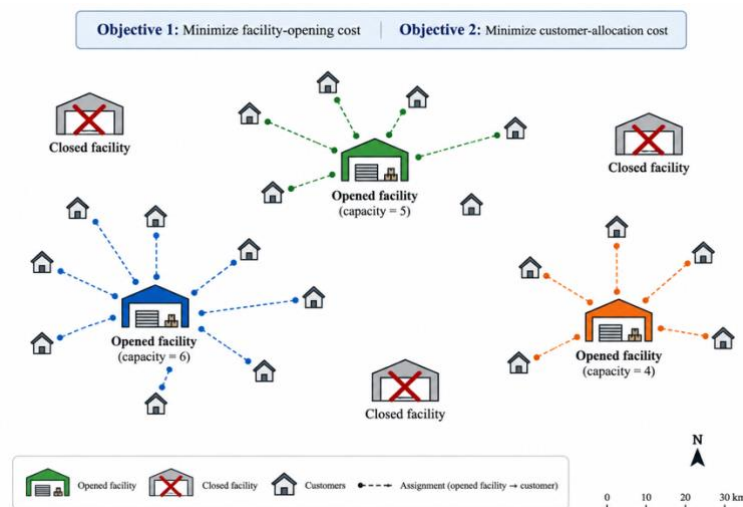


Fig. 1: Illustration of the multi-objective CFLP.

Unlike a single-objective formulation that combines all costs into a single number, this assignment keeps the two cost components separate. This allows the MOEA to generate a Pareto set of trade-off solutions: some solutions may use fewer or cheaper facilities but incur a higher customer-allocation cost, while others may open more facilities to reduce allocation cost.

Let $i = 1, \dots, m$ denote candidate facilities and $j = 1, \dots, n$ denote customers. Let $y_i = 1$ if facility i is opened (0 otherwise), and $x_{ij} = 1$ if customer j is assigned to facility i (0 otherwise). The two objectives that must be optimized simultaneously are:

- **Objective-1:** Facility-opening cost: $\min f_1 = \sum_i F_i y_i$
- **Objective 2** — Customer-allocation cost: $\min f_2 = \sum_i \sum_j C_{ij} x_{ij}$

where F_i is the fixed cost of opening facility i and C_{ij} is the cost of allocating all demand of customer j to facility i . Therefore, C_{ij} already includes the effect of customer demand and **must not be multiplied by d_j again** when calculating Objective 2.

Mandatory feasibility constraints:

- Every customer is assigned exactly once: $\sum_i x_{ij} = 1$ for every customer j
- Assignment only to an opened facility: $x_{ij} \leq y_i$
- Facility capacity: $\sum_j d_j x_{ij} \leq S_i y_i$ for every facility i

Here d_j is customer demand and S_i is facility capacity. Both objective values must be calculated only after a feasible customer assignment has been produced.

Dataset Source

Use the public OR-Library Capacitated Warehouse Location benchmark instances maintained by J. E. Beasley. Dataset information page: <https://people.brunel.ac.uk/~mastjjb/jeb/orlib/capinfo.html> ; Data files: <https://people.brunel.ac.uk/~mastjjb/jeb/orlib/files/>

Project quantity	Data used	Exact source in OR-Library instance
f_1 : Facility-opening cost	Fixed opening cost F_i	For each candidate facility: the “fixed cost” field.
f_2 : Customer-allocation cost	Allocation cost C_{ij}	For each customer: the cost of allocating all of that customer’s demand to each facility. Do not multiply this value by customer demand again.
Feasibility constraints	Facility capacity S_i and customer demand d_j	Facility “capacity” field and each customer “demand” field.

Important: You must NOT invent, randomly generate, or replace the fixed costs, allocation costs, capacities, or customer demands. Use the OR-Library values directly. Supply the same input instance to both MOEAs.

Required Benchmark Instances

Use six OR-Library instances, grouped by the number of candidate facilities.

Category	Required instances	Candidate facilities (m)	Customers (n)
Small	<i>cap41</i> , <i>cap42</i>	16	50
Medium	<i>cap101</i> , <i>cap102</i>	25	50
Large	<i>cap121</i> , <i>cap122</i>	50	50

Solution Representation and Feasibility Handling

You may design your own representation, but you must use the SAME representation and the SAME problem-specific variation/repair logic in both of your chosen MOEAs so that the algorithm comparison is fair. If your designed crossover and mutation operators create infeasible solutions, your implementation must include an explicit feasibility-handling procedure.

MOEAs and Fair Comparison:

You may use standard numerical/data libraries, but your code should make the evolutionary logic and main algorithmic issues visible (i.e., do not use library functions for core operations). For example, if you choose NSGA-II, you should implement non-dominated sorting, crowding distance, tournament selection, variation, mutation, and elitist environmental selection in your code. For SPEA-2, these include strength/raw fitness, density estimation, external archive, truncation, selection, mutation, and variation.

For a fair comparison, evaluate both algorithms under the same experimental conditions. You must define three parameter configurations. Within each configuration, both MOEAs must use the same parameter configuration, such as population size, initialisation method, crossover and mutation probabilities, termination criterion, and maximum number of objective evaluations. The parameter values may differ between the three configurations.

Experimental Design and Multi-Objective Performance Metrics

Because Evolutionary Algorithms are stochastic, **single-run results are unacceptable**. You **must run each experiment for at least 10 independent runs** using the same computational budget/common parameters. For each algorithm/instance combination, report at least the following:

- Hypervolume (HV): primary indicator of convergence and coverage. Larger is better.
- Number of non-dominated solutions in the final approximation set.
- Execution time in seconds.
- Final Pareto-front scatter plot (f_1 against f_2), showing both MOEAs on the same axes.

Hypervolume calculation: Both MOEAs must use the same objective scaling/normalization procedure and the same Hypervolume reference point for each benchmark instance. For this minimization problem, the reference point must represent objective values worse than all solutions included in the comparison. Clearly describe the scaling/normalization procedure and reference point in the report.

Using the metrics above, compare and analyze both MOEAs across any three benchmark instances (one each from small, medium, and large instances) and **the three parameter configurations defined above**. Analyse and report how the parameter configurations affect Pareto-front convergence, diversity, and computational time.

Deliverables

- a. **Code Implementation:**
 - i. Provide **one shareable** GitHub repository containing clean Python code, executable scripts, step-by-step installation instructions in README .md, and inline documentation explaining your algorithmic logic. The README must include instructions for installing any dependencies and running the algorithms on the provided example data.
 - ii. **The code must be executable and generate the outputs.**

- iii. The code must load the required OR-Library data files and clearly separate data loading, chromosome representation, feasibility repair/decoding, objective evaluation, metrics, statistical tests, and plotting.
- iv. The repository must contain executable scripts/notebooks that reproduce the reported experiments and figures.
- v. Use fixed/reported random seeds so experiments can be reproduced. Do not submit only generated results without executable code.
- vi. **Include the GitHub link in the submitted report and ensure it is shareable and executable.**

b. Report

- i. You should deliver your report in a PDF format through Canvas.
- ii. The project report should be between 1000-1500 words.
- iii. **The report must mention the Group No as published in the Canvas.**
- iv. The report should at least discuss the points below:
 1. A clear real-world description of the CFLP and a mathematical definition of both optimization objectives and all constraints.
 2. Using one example encoded individual, describe how your representation and repair/decoding procedure produces a feasible facility-opening and customer-assignment solution..
 3. A clear statement of the data source for Objective 1 and Objective 2, identifying the exact OR-Library fields used for each objective.
 4. A detailed description of the complete MOEA design. Describe your implementation in sufficient detail for others to replicate your work. This includes every algorithmic (for example, in the case of NSGA-II: non-dominated sorting, crowding distance, tournament selection, etc.) and genetic operator (for example, individual representation, initialization strategy, selection strategy, crossover and mutation operators, termination condition) for your chosen MOEAs.
 5. A concise but replicable description how the multi-objective selection/environmental replacement differs for the chosen algorithms.
 6. The exact common experimental budget and parameter settings used to ensure a fair comparison.
 7. Using the specified metrics, compare and analyze your implementation across three categories of test problems.
 8. For every selected instance, provide a table reporting the mean, standard deviation, best and worst Hypervolume (HV), number of non-dominated solutions, and mean execution time over the required independent runs.
 9. Three Pareto-front plots (one each from small, medium, and large instances) comparing the two.
 10. Analysis of the three parameter configurations and how they affect convergence, diversity, and runtime.